

© 2024 Zirui Wang

VIDECOMP: VIDEO QUESTION ANSWERING BY COMPOSITIONAL QUERY

BY

ZIRUI WANG

THESIS

Submitted in partial fulfillment of the requirements
for the degree of Bachelor of Science in Computer Science
with a minor in Electrical and Computer Engineering
in the Grainger Engineering College of the
University of Illinois Urbana-Champaign, 2024

Urbana, Illinois

Advisor:

Professor Prof Lazebnik

Abstract

The rapid proliferation of video content in various domains underscores a burgeoning need for sophisticated systems capable of comprehending and summarizing long videos to facilitate efficient information retrieval and question answering (QA). Traditional methods, focusing on frame-by-frame analysis, often fall short in capturing temporal dynamics and lack the ability to provide a coherent summary of the video content. Moreover, procedural reasoning frameworks such as VISPROG and ViperGPT leverage Large Language Models (LLMs) agents to understand complex environment and have shown to be successful in image level tasks such as image question answering. However, these models still operate on individual frames, restricts the exploitation of temporal relationships across frames, thus hindering the accuracy and richness of video understanding. In this work, we presents Video Understanding via Compositional Query (**VideoComp**), an extended version of ViperGPT which combines video modules and image modules for video understanding, achieving stronger generalization ability over a wide variety of videos. Evaluating on Video Question Answering (Video-QA) benchmarks, the proposed framework brings about 3% improvement on general accuracy with less visual modules enabled in the process. In addition, we have shown the effectiveness of each enabled visual modules with extensive experiments.

Acknowledgments

This project would not have been possible without the support of many people. Many thanks to my adviser, Svetlana Lazebnik, who always been professional and responsible on my thesis research and offered guidance and support. Thanks to my amazing two mentors, Sethuraman T V for his guidance on this project and being a super supportive mentor, and Daniel McKee for his continuous support on the project and providing interesting high level ideas. Finally, thanks to my family who endured this long process with me, always offering support and love.

Table of contents

Chapter 1	Introduction	1
Chapter 2	Related Work	3
Chapter 3	Method	5
Chapter 4	Experiments	8
Chapter 5	Conclusions	12
References		13
Appendix A	API Documentation	16
Appendix B	More Examples	19

Chapter 1

Introduction

The rapid proliferation of video content in various domains underscores a burgeoning need for sophisticated systems capable of comprehending and summarizing long videos to facilitate efficient information retrieval and question answering (QA).

At earlier stage, many works on video understanding focus more on training a supervised model using large video datasets such as Ego4D [1] and fine-tune on smaller datasets subsequently, but it fails at generalize to other domain of interests. Other traditional methods, focusing on frame-by-frame analysis, often fall short in capturing temporal dynamics and lack the ability to provide a coherent summary of the video content. Recently, the static nature of current neuro-symbolic reasoning frameworks like VISPROG [2] and ViperGPT [3] leverage LLMs to generate procedural programs that solve the compositional image tasks and has some success to it. In particular ViperGPT also demonstrate strong potential to propagate from image tasks to video tasks. However, ViperGPT still operates on individual frames, restricts the exploitation of temporal relationships across frames, thus hindering the accuracy and richness of video understanding.

Addressing this problem is pivotal as it stands to significantly advance the fields of video understanding, information retrieval, and QA systems. A robust solution has the potential to transform myriad sectors including surveillance, healthcare, education, and media by enabling rapid, accurate, and contextually rich analysis of video content. This would markedly reduce manual labor and time required for video analysis, expedite decision-making processes, and pave the way for real-time monitoring and interactive video-based applications.

Recognizing the significance of the problem, we propose a Video Question Answering by Compositional Query (VideoComp), a unified framework that leverages LLMs to generate python executable code to answer complex queries. Taking advantage of the strong capability of large language models including GPT-3.5 [4] and GPT-4 [5], we provide LLMs an prompt that includes visual module APIs as well as some in-context examples to generate code that answers the questions. We include a combination of image and video models that enable us to acquire meaningful information from low level semantic information, to high-level video summaries that help us to better solve the problem.

Our design of compositional reasoning has several advantage over traditional methods and existing works. To start with, VideoComp runs in a zero-shot manner, no extra training is necessary. Given that the visual

modules use pre-trained models trained on large datasets, it largely guarantee the generalizability of these visual modules when solving simple queries. Besides, module composition is flexible, where LLM can generate any combination of visual modules depending on the query itself. With the aforementioned advantages, we manage to apply it to large video benchmark and results in an 2.6% improvement over NeXT-QA benchmark.

To sum up, our paper has the following contributions:

1. We present VideoComp, a method to step-by-step reason about and answer zero-shot video queries through generated Python programs.
2. We achieve accuracy improvements on a major video QA benchmarks NExT-QA and provide extensive ablations to demonstrate the utility of our approach.

Chapter 2

Related Work

2.1 Modular Vision

The concept of modular vision is first proposed in Neural Modular Networks (NMNs) [6], where the authors argue that complex vision reasoning tasks are fundamentally compositional so that it can be divided into atomic perceptual units for modular processing. This visual reasoning procedure has been explored by a variety of works [7], [8]. Posterior works [9]–[12] have explored other possibilities that require supervised learning methods to train the perception module in an end-to-end manner. Though it helps with performance on specific tasks, however, this group of models results in a large domain gap that is hard for these supervised models to generalize to solve other downstream tasks.

In the past year, Large Language Models (LLMs) play a significant role in complex reasoning tasks. Leveraging on its strong zero-shot or few-shot inference ability, some works [2], [3], [13] have shown success in using language models to generate modular processing pipelines that reformulate the compositional reasoning task through code generation: they used language models with strong performance such as GPT-3 [4] and GPT-4 [5] to generate answers as short Python programs, together using Python to build in mathematical operations, if-else statements as well as logical operators to process the output from the visual modules. ViperGPT results in strong performance on the NeXT-QA dataset [14], but is still limited by its temporal reasoning ability for further experiments. To overcome the limitation on temporal reasoning, we directly build off ViperGPT’s approach with the same program generation framework and manage to extend it to video level question answering tasks.

2.2 Pretrained Models

With the emergence of large language models, many papers have studied how to effectively leverage it as a reasoning agent to solve downstream tasks, either improving performance and generalizability through fine-tuning [15] or prompting [16]–[18]. Inspired by the multi-modal models like Flamingo [19], recent methods explore the possibility of training multi-modal language models, including successful examples such as MiniGPT-4 and its variants [20], [21] and LLaVA [22], [23] that add visual capabilities to the language model, empowering a new dimension to the existing language models. However, the above-mentioned models failed to incorporate video inputs, due to challenges of training on large-scale video data.

With these powerful pretrained models, people start build on top of it by combining the capability of multiple foundation models into a unified framework such as Visual chatGPT [24]. Following VISPROG and ViperGPT, we use the combination of pretrained models on image and videos for visual functions like image QA and video object tracking detection and image QA. For image pretrained model, we use BLIP-2 [25] for image-conditioned QA. For video pretrained model, we use Video-LLaVA [26] for video-to-language (including image-to-language) generation and DEVA [27] for multi-object tracking. As for the core large language model for reasoning, we use GPT-3.5 [4] for generating code and querying summaries.

2.3 Video Question Answering (Video-QA)

Using notations from [28], video question answering, or Video-QA, is a task to predict the correct answer a^* based on a question q and a video input V . Generally there are two types of tasks: multichoice QA and open-ended QA. In this work, we only focus on multichoice QA.

Compared to other video tasks, such as video object tracking and video captioning, video question-answering datasets are relatively small. A predominant pipeline in Video QA task is to pre-train models on several large internet scale datasets such as Ego4D [1], HowTo100M [29] and HowToVQA69M [30], and fine-tuned on smaller specifically annotate QA dataset [28], [31]. However, these models still have limited performance at zero-shot question answering, failed to generalize to other domains outside of the training data distribution.

More recent works tries to approach zero-shot video question answering and mainly there are two directions. On one hand, some methods try to jointly train models with multi-modality, especially on web-scale datasets in a weakly supervised manner using audio and text scripts [30], [32], [33]. These methods obtains the state-of-the-art results on Video Question Answering. On the other hand, other works has leverage more on the strong reasoning ability from LLMs and combine it with visual inputs to answer the query, but since LLMs aren't stable enough to generate information relevant to the query, these noises would confuse the language model and affect the prediction significantly.

Chapter 3

Method

Following [3], given a visual input x and a text-based query q as the input, we first generate a program $z = \pi(q)$ with a program generator π given only the query q . Then, with the generated code, we feed the code script in to a execution engine $r = \phi(z, x)$ that execute the program z on the visual input x . The result after the execution of program is stored in r . Noted that even though the framework is designed for video level reasoning, visual input x supports both video input (lists of frames) and single image input. In the following sections, we will introduce each steps in details, including program generation, visual module designs, prompting and examples.

3.1 Program Generation

The input to the VideoCQ is an video and the query, where the query can be a open-ended question, text descriptions and etc. Then, we constructed the prompt for code generator, which includes the input query, an application programming interface (API) containing the list of available visual modules that help us to solve the queries, and a few in-context example programs based on the API. With the integrated prompt, we fed to the LLM agent to generate executable python program. Here, we use gpt-3.5-turbo, a variant of [4], for all experiments. The expected output from the LLM is a short Python program that break down the complex query into several steps, each consisting of a simplified query that can be processed by one of the visual modules defined in the API. Once the code output is generated, we then can compile an execute similar to [3] that uses the Python’s build-in `exec()` function.

3.2 Video Modules and APIs

We define and describe a list of video modules in the provided API, which are specifically designed as a toolbox for programs to break down the complex query into simpler ones and help answer the question. Each module is defined to achieve the best generalizability as we want LLMs to experiment on different combination of these videos modules to answer the query in the most logical and effective way. It is likely that the API isn’t as exhaustive as [3], but with fewer visual modules, we find each video module effective to solve the given query, which also help with predicting the correct answer. In addition, the major pipeline of solving complex video understanding queries involves down-sampling key-frames from the collections of video frames and combining visual information from the key-frames with high-level contextual information to

uses an LLM (gpt-3.5-turbo for all experiments) to answer a multiple choice question by choosing the best answer among the possible answers. The context information can be visual clues, such as caption of videos and images, or textual, such as a summary of the input video or even a smaller segment of it. This method is crucial for solving multiple choice benchmarks.

5. **visual_query**: Different from `language_query`, this method invokes BLIP-2’s QA capability to answer a question about collections of frames. For example, given the query ”what is the man doing?” and the possible answers, we concatenate the query to the possible answers and fed into BLIP-2 for frame-by-frame analysis. To minimize the influence from outliers, we vote over all possible answers and choose the one with the highest vote as the final output prediction.

3.3 In-Context Examples

Recent literature [18], [34] has shown the importance of careful wording of the input prompt and that in-context examples greatly affect LLMs’ performance for downstream tasks. ViperGPT uses same example programs in method documentation across different benchmarks, which may potentially lead to bias at code generation steps. Therefore, we construct in-context examples following VISPORG [2] and CodeVQA [13] where we come up with sample programs relevant to the dataset itself. In addition, for LLM-based module such as `visual_query`, we keep the prompts fixed as ViperGPT does. We provide detailed ablation studies on prompt components later in the paper.

Chapter 4

Experiments

4.1 Experiment Setup

Implementation Detail Our code base is a heavily modified version of the ViperGPT [3] codebase, with some improvement on efficient processing, such as multi-processing and removal of unnecessary modules. We then add all our visual module implementation on top of it. Similar to ViperGPT, we constantly make API calls to GPT-3.5 [4], with function definitions and several in-context examples defined in the prompt. Prompt is a text file, which is different from one another based on the dataset context. For visual modules, we use BLIP-2 for Image level Question Answering (QA), DEVA for video level object detection and tracking (with Xmem [35] as backbone tracker), and Video-LLaMA for video captioning. We obtain different accuracy over baseline ViperGPT model due to the usage of smaller version (with fewer model parameters) of BLIP-2 and Video-LLaMA in order to load all model onto a single Nvidia A40 GPU, but we keep the same setting across different experiments.

Dataset We evaluate the framework mainly on NeXT-QA [14] datasets, comparing it with the baseline model ViperGPT [3]. NeXT-QA is a multiple-choice benchmark designed to test causal and temporal reasoning. Each question has 5 distinct choices and it typically requires the combination of high-level contextual information and low-level visual information to correctly answer the question.

Metric We use top-1 accuracy as the only evaluation metric here given that multi-choice QA dataset exist a single correct answer, even though the questions are often vaguely phrased and have multiple valid answers.

4.2 Video QA Results

Model	Temporal	Causal	All
ViperGPT [3]	46.7	51.4	55.1
VideoComp(Ours)*	51.3	54.1	57.7

Table 4.1: **NeXT-QA Results:** We report zero-shot accuracy on the test split. We display the detailed top-1 accuracy per category.

Query: "how does the man in white prevent the white and black dog from running away?"

Code Generation:

```
def execute_command(video, possible_answers, query, VideoSegment):
    # Answer is def execute_command(video, possible_answers, query):
    video_segment = VideoSegment(video)
    # Reason step-by-step
    man_segment = video_segment.object_sampling("man in white")
    dog_segment = man_segment.object_sampling("dog")
    prevent_segment = dog_segment.general_sampling("Does the man prevent the dogs from running away?")
    # create context dictionary
    image_caption = prevent_segment.get_caption(prevent_segment.num_frames//2)
    video_caption = prevent_segment.get_caption()
    context = {
        "Description of a scene when man prevents the dogs from running away": image_caption,
        "Description of the video segment ": video_caption
    }
    # Answer the query
    answer = prevent_segment.language_query(context, query, possible_answers)
    return answer
```

Video:



video_segment.object_sampling("man in white")



man_segment.general_sampling("Does the man prevent the dogs from running away?")



prevent_segment.get_caption(prevent_segment.num_frames // 2)



image_caption = "In the image, two dogs are standing next to each other on a sidewalk. One of the dogs has a red leash, while the other dog is wearing a blue leash."

prevent_segment.get_caption()

video_caption = "At the beginning of the video, we see a man walking his dog down the street. Then, we see a woman walking her dog on a path. Next, a man and a woman are walking their dogs on the street, and they are followed by a man and a woman who are walking their dog down a street. Then, we see a man and a woman walking their dog in the street, and they are followed by a woman walking her dog. Finally, the video ends with a man walking his dog down the street."

Prediction: "use collar strap"

Figure 4.1: The example walkthrough step-by-step reasoning procedure taken by **VideoComp**. For each step, we visualize the intermediate output to see how our framework manage to down-sample the size of frames and use only relavent information to solve comptex question in zero-shot manner.

Our main experiment results are displayed in Table 4.1. We have achieved an accuracy improvements on multiple choice benchmarks, with particularly gains in 2.7% on the whole test split on NeXT-QA benchmark. We also include zero-shot question answering results per-category on the NeXT-QA dataset: we outperform ViperGPT on the top-1 accuracy score for 2.7% on causal queries and 4.6% on temporal queries. By visually verifying with experimental results, we attribute the improvement to a number of factors. Queries from NeXT-QA dataset require knowledge from context information given that causal queries cannot be answered directly by a single frame. However, ViperGPT mainly uses image-centric API and is unable to pay attention to the larger contextual information, so it is unable to perform reasoning over long time intervals, but our framework incorporate video captioning and object tracking models which ensure consistency in temporal space, while maintaining strong image-level visual models as another source for visual information. Noted that the improvement over baseline on NeXT-QA benchmark is limited due to the constraints on the number of choices comparing to open-ended QA datasets.

4.3 Ablation Studies

Effectiveness of Visual Modules Here we carry out experiment evaluating the usefulness of each individual visual modules in the line of reasoning and observed an boost in performance when adding more and more visual modules. The complete result is shown in 4.2. We first start off with a single image QA model, where we run BLIP-2 [25] over the middle frame in the video and select the answer. From the table, we can see that a single image QA model result in a fairly strong baseline, suggesting that a image QA model with a easy question is helpful. Adding two frame down-sampling functions, `object_sampling` and `general_sampling`

adds reasoning capability over video clips and result in improvement on top-1 accuracy, suggesting that video level reasoning module help reduce the error from the single frame reasoning and achieving consensus across the video. In the given context, these two visual modules help NeXT-QA dataset by downsampling key frames to filter out irrelevant frame information that may contribute as noise to the model. Finally, `get_caption` module adds a significant improvement on the dataset, since visual descriptions in video is important for answering causal questions.

Visual Modules				Accuracy (%)
<code>image_QA</code> (baseline)	✓			46.7
<code>object_sampling</code>	✓			50.1
<code>general_sampling</code>	✓	✓		52.9
<code>get_caption</code>	✓	✓	✓	55.4
ALL	✓	✓	✓	57.8

Table 4.2: **Ablation Study on Visual Modules:** We perform ablation study on each major visual modules including `visual_query`, `object_sampling`, `general_sampling`, `get_caption` and the full prompt on NeXT-QA dataset. We gradually add new visual modules to the prompt and observe an improvement for every module.

# of Examples	Accuracy (%)
0 example	21.3
1 example	49.8
3 example	53.7
6 example	56.9

Table 4.3: **Ablation Study on In-context Examples:** We evaluate the impact of number of in-context examples on model performance: when there’s no in context example, the model performs poorly, but gradually adding more in-context example boost the performance

In-context Examples We also evaluate on effect of number of in-context examples on LLMs’ ability to generate reasonable code, and result is shown in 4.3. Clearly, with no in-context example, the language model, GPT-3.5 in our experiment, perform poor on generating useful code. By looking at actual examples code generations, we observe that the large language model usually generate random code that failed to compile. Adding one in-context example gives LLM some hint on generating effective code, but failure cases fall in one of the two following cases: 1) the code failed to compile; 2) the LLM generate exactly same code as the only in-context example. Adding more in-context examples gradually help boost the performance. Based on several experiments, we find that usually 4 - 6 in context examples result in best performance.

4.4 Other Failure Experiments Analysis

Query Conditioning: Other than the main experiments above, we try providing extra contextual information to the query prior to the code generation, expecting such context can be critical to correctly responding to a wide range of queries. In experiments, we sampled an subset of 500 samples from NeXT-QA dataset and use Video-LLaMA [26] for video captioning, where we query the model to generate a summary over the entire video, and we then fed the video summary as additional contextual information to the LLM for code generation. As the example shown in Figure 1, the video summary provide extra information to LLM and generate different code than before, but careful examination over the new code generation would we observe

that the code generation has been deviated from the query and focus on the irrelevant information introduced by the video summary, therefore resulting in an total 9.2% drop on top-1 accuracy as shown in table 4.4

Model	Accuracy (%)
ViperGPT [3]	0.52
ViperGPT w/ cond.	0.43

Table 4.4: **Conditioning Results:** We report zero-shot accuracy on the test split. Notice that the performance drops when conditioning is introduced to the ViperGPT model.

Chapter 5

Conclusions

We presented VideoComp, a method that extends modular vision methods to zero-shot video question answering. Our approach achieves a significant improvement over the state-of-the-art by using strong language models for temporal reasoning with code rather than using an end-to-end network. Though the experiment setup is limited by resources, we have shown proof of concept that video compositional query would be helpful in solving complex reasoning tasks. Alongside, ablation studies and analysis demonstrate the strengths of our method, suggesting procedural reasoning can improve performance for vision tasks in general. We believe VideoComp will serve as a strong video question answering baseline for the community to be a step forward.

References

- [1] K. Grauman, A. Westbury, E. Byrne, *et al.*, *Ego4d: Around the world in 3,000 hours of egocentric video*, 2022. arXiv: [2110.07058](https://arxiv.org/abs/2110.07058) [cs.CV].
- [2] T. Gupta and A. Kembhavi, “Visual programming: Compositional visual reasoning without training,” *ArXiv*, vol. abs/2211.11559, 2022.
- [3] D. Surís, S. Menon, and C. Vondrick, “Vipergpt: Visual inference via python execution for reasoning,” *arXiv preprint arXiv:2303.08128*, 2023.
- [4] T. Brown, B. Mann, N. Ryder, *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33, Curran Associates, Inc., 2020, pp. 1877–1901. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf.
- [5] OpenAI, J. Achiam, S. Adler, *et al.*, *Gpt-4 technical report*, 2024. arXiv: [2303.08774](https://arxiv.org/abs/2303.08774) [cs.CL].
- [6] J. Andreas, M. Rohrbach, T. Darrell, and D. Klein, “Neural module networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2016.
- [7] S. W. Kim, M. Tapaswi, and S. Fidler, “Visual reasoning by progressive module networks,” in *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*, OpenReview.net, 2019. [Online]. Available: <https://openreview.net/forum?id=B1fpDsAqt7>.
- [8] S. Whitehead, H. Wu, H. Ji, R. Feris, and K. Saenko, “Separating skills and concepts for novel visual question answering,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 5632–5641.
- [9] K. Yi, J. Wu, C. Gan, A. Torralba, P. Kohli, and J. B. Tenenbaum, “Neural-symbolic vqa: Disentangling reasoning from vision and language understanding,” in *Advances in Neural Information Processing Systems*, 2018, pp. 1039–1050.
- [10] R. Girdhar, A. El-Nouby, Z. Liu, *et al.*, “Imagebind: One embedding space to bind them all,” in *CVPR*, 2023.
- [11] Z. Wang, M. Li, R. Xu, *et al.*, “Language models with image descriptors are strong few-shot video-language learners,” *arXiv preprint arXiv:2205.10747*, 2022.
- [12] R. G. Reddy, X. Rui, M. Li, *et al.*, “Mumuqa: Multimedia multi-hop news question answering via cross-media knowledge extraction and grounding,” *arXiv preprint arXiv:2112.10728*, 2021.
- [13] S. Subramanian, M. Narasimhan, K. Khangaonkar, *et al.*, “Modular visual question answering via code generation,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, Toronto, Canada: Association for Computational Linguistics, 2023.

- [14] J. Xiao, X. Shang, A. Yao, and T.-S. Chua, “Next-qa: Next phase of question-answering to explaining temporal actions,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2021, pp. 9777–9786.
- [15] T. Schick, J. Dwivedi-Yu, R. Dessì, *et al.*, *Toolformer: Language models can teach themselves to use tools*, 2023. arXiv: [2302.04761](#) [cs.CL].
- [16] K. Li, Y. He, Y. Wang, *et al.*, “Videochat: Chat-centric video understanding,” *arXiv preprint arXiv:2305.06355*, 2023.
- [17] Z. Yang, Z. Gan, J. Wang, *et al.*, “An empirical study of gpt-3 for few-shot knowledge-based vqa,” in *AAAI*, 2022.
- [18] J. Wei, X. Wang, D. Schuurmans, *et al.*, *Chain-of-thought prompting elicits reasoning in large language models*, 2023. arXiv: [2201.11903](#) [cs.CL].
- [19] J.-B. A. *et al.*, “Flamingo: A visual language model for few-shot learning,” 2022.
- [20] D. Zhu, J. Chen, X. Shen, X. Li, and M. Elhoseiny, “Minigpt-4: Enhancing vision-language understanding with advanced large language models,” *arXiv preprint arXiv:2304.10592*, 2023.
- [21] J. Chen, D. Zhu, X. Shen, *et al.*, “Minigpt-v2: Large language model as a unified interface for vision-language multi-task learning,” *arXiv preprint arXiv:2310.09478*, 2023.
- [22] H. Liu, C. Li, Q. Wu, and Y. J. Lee, *Visual instruction tuning*, 2023.
- [23] H. Liu, C. Li, Y. Li, *et al.*, *Llava-next: Improved reasoning, ocr, and world knowledge*, Jan. 2024. [Online]. Available: <https://llava-vl.github.io/blog/2024-01-30-llava-next/>.
- [24] C. Wu, S. Yin, W. Qi, X. Wang, Z. Tang, and N. Duan, *Visual chatgpt: Talking, drawing and editing with visual foundation models*, 2023. arXiv: [2303.04671](#) [cs.CV].
- [25] J. Li, D. Li, S. Savarese, and S. Hoi, “BLIP-2: bootstrapping language-image pre-training with frozen image encoders and large language models,” in *ICML*, 2023.
- [26] H. Zhang, X. Li, and L. Bing, “Video-llama: An instruction-tuned audio-visual language model for video understanding,” *arXiv preprint arXiv:2306.02858*, 2023. [Online]. Available: <https://arxiv.org/abs/2306.02858>.
- [27] H. K. Cheng, S. W. Oh, B. Price, A. Schwing, and J.-Y. Lee, “Tracking anything with decoupled video segmentation,” in *ICCV*, 2023.
- [28] Y. Zhong, J. Xiao, W. Ji, Y. Li, W. Deng, and T.-S. Chua, “Video question answering: Datasets, algorithms and challenges,” in *The 2022 Conference on Empirical Methods in Natural Language Processing*, 2022.
- [29] A. Miech, D. Zhukov, J.-B. Alayrac, M. Tapaswi, I. Laptev, and J. Sivic, “HowTo100M: Learning a Text-Video Embedding by Watching Hundred Million Narrated Video Clips,” in *ICCV*, 2019.
- [30] A. Yang, A. Miech, J. Sivic, I. Laptev, and C. Schmid, “Just ask: Learning to answer questions from millions of narrated videos,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 1686–1697.
- [31] D. Huang, P. Chen, R. Zeng, Q. Du, M. Tan, and C. Gan, “Location-aware graph convolutional networks for video question answering,” in *AAAI*, 2020.

- [32] A. Yang, A. Miech, J. Sivic, I. Laptev, and C. Schmid, “Zero-shot video question answering via frozen bidirectional language models,” in *NeurIPS*, 2022.
- [33] R. Zellers, X. Lu, J. Hessel, *et al.*, “Merlot: Multimodal neural script knowledge models,” in *Advances in Neural Information Processing Systems 34*, 2021.
- [34] P. Lewis, E. Perez, A. Piktus, *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33, Curran Associates, Inc., 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.
- [35] H. K. Cheng and A. G. Schwing, “XMem: Long-term video object segmentation with an atkinson-shiffrin memory model,” in *ECCV*, 2022.

Appendix A

API Documentation

```
1 class VideoSegment:
2     """A Python class containing a set of frames represented as ImagePatch objects, as well
3     as relevant information.
4     Attributes
5     -----
6     video : torch.Tensor
7         A tensor of the original video.
8     start : int
9         An int describing the starting frame in this video segment with respect to the
10        original video.
11    end : int
12        An int describing the ending frame in this video segment with respect to the
13        original video.
14    num_frames->int
15        An int containing the number of frames in the video segment.
16    """
17
18    def __init__(self, video: torch.Tensor, start: int = None, end: int = None, parent_start
19    =0, queues=None):
20        """
21        Initializes a VideoSegment object by trimming the video at the given [start, end]
22        times and stores the
23        start and end times as attributes. If no times are provided, the video is left
24        unmodified, and the times are
25        set to the beginning and end of the video.
26
27        Parameters
28        -----
29        video : torch.Tensor
30            A tensor of the original video.
31        start : int
32            An int describing the starting frame in this video segment with respect to the
33            original video.
34        end : int
35            An int describing the ending frame in this video segment with respect to the
36            original video.
37        """
```

```

31 def trim(self, start: Union[int, None] = None, end: Union[int, None] = None) ->
VideoSegment:
32     """
33     Returns a new VideoSegment containing a trimmed version of the original video at the
[start, end]
34     segment.
35
36     Parameters
37     -----
38     start : Union[int, None]
39         An int describing the starting frame in this video segment with respect to the
original video.
40     end : Union[int, None]
41         An int describing the ending frame in this video segment with respect to the
original video.
42
43     return : VideoSegment
44         A new VideoSegment containing a trimmed version of the original video at the [
start, end] segment.
45
46     """
47
48 def object_sampling(self, object_name: str) -> VideoSegment:
49     """
50     Returns a new VideoSegment containing a sampled version of the original video with
only frames
51     which object_name presents.
52
53     Parameters
54     -----
55     object_name : str
56         A string containing the name of the object to sampling.
57
58     Returns: VideoSegment
59         A new VideoSegment containing the frames with the object or None if the object
is not detected.
60
61     Examples
62     -----
63     # how many cats are there in the video?
64     def execute_command(video, possible_answers):
65         video_segment = VideoSegment(video)
66         cat_segment = video_segment.object_sampling("cat")
67         answer = cat_segment.visual_query("how many cats are there in the video?",
possible_answers)
68         return answer
69     """
70
71 def general_sampling(self, general_inquiry: str) -> VideoSegment:
72     """
73     Returns a new VideoSegment containing a sampled version of the original video with
only frames
74     which the statement in general_inquiry holds True/yes.
75

```

```

76     Parameters
77     -----
78     general_inquiry : str
79         A string containing statement that sampled video segment.
80
81     Returns: VideoSegment
82         A new VideoSegment containing the frames with the statement in the
83         general_inquiry is true.
84     """
85
86     def get_caption(self, idx=None) -> str:
87         """
88         Returns a captioning of the video. If idx is provided, the summary is generated from
89         frame idx in the video, otherwise, the summary is generated from the video.
90
91         Parameters
92         -----
93         idx : int
94             A int describing the frame in this video segment with respect to the original
95             video.
96
97         Returns: str
98             A string representation of the the narratives of the video or the image frame
99
100         """
101
102     def language_query(self, context: dict, question: str, options=None) -> str:
103         """
104         Answer a question ONLY from context. If options are provided,
105         the answer is selected from the options.
106
107         Parameters
108         -----
109         context : dict
110             A dictionary containing the context information.
111         question : str
112             A text question to ask.
113         options : list
114             A list containing the possible answers to the question.
115
116         Returns: str
117             A string containing the answer to the question.
118         """

```

Listing A.1: API Documentation

Appendix B

More Examples

Here we includes more code generation examples for the NeXT-QA dataset.

```
1 # why did the lady in blue capture the scene with her camera when the man is riding the bull
2 # possible_answer: ['requested by the man', 'the man is nearing the finishing line', 'keep
   for memory', 'it was the first time seeing the bull', 'submit for a video competition']
3 def execute_command(video, possible_answers, query, VideoSegment):
4     # Answer is:
5     def execute_command(video, possible_answers, query):
6         video_segment = VideoSegment(video)
7         # Reason step-by-step
8         man_segment = video_segment.object_sampling("man")
9         bull_segment = man_segment.object_sampling("bull")
10        lady_segment = bull_segment.object_sampling("lady in blue")
11        camera_segment = lady_segment.general_sampling("Is the lady capturing the scene with her
   camera?")
12        # create context dictionary
13        image_caption = camera_segment.get_caption(camera_segment.num_frames//2)
14        video_caption = camera_segment.get_caption()
15        context = {
16            "Description of a scene when the lady captures the scene with her camera":
17            image_caption,
18            "Description of the video segment ": video_caption
19        }
20        # Answer the query
21        answer = camera_segment.language_query(context, query, possible_answers)
22        return answer
```

Listing B.1: Causal Reasoning Example in NeXT-QA dataset

```
1 # Query: what did the man do before he cut the fish sideways
2 # possible_answer: ['wriggle around', 'interested', 'cut fish head', 'pull it up', 'moved
   its body']
3 def execute_command(video, possible_answers, query, VideoSegment):
4     # Answer is:
5     def execute_command(video, possible_answers, query):
6         video_segment = VideoSegment(video)
7         # Reason step-by-step
8         man_segment = video_segment.object_sampling("man")
9         fish_segment = man_segment.general_sampling("Is the man cutting a fish sideways?")
10        # create context dictionary
11        image_caption = fish_segment.get_caption(fish_segment.num_frames//2)
```



```

11 video_caption = fish_segment.get_caption()
12 context = {
13     "Description of a scene when the man cutting the fish sideways": image_caption,
14     "Description of the video segment ": video_caption
15 }
16 # Answer the query
17 answer = fish_segment.language_query(context, query, possible_answers)
18 return answer

```

Listing B.2: Temporal Reasoning Example in NeXT-QA dataset

```

1 # how does the baby react to the man moving forward and backward at the start
2 # possible_answer: ['he goes out', 'lie down', 'laugh and smile', 'he throws a tantrum', 'he
  starts to cry']
3 def execute_command(video, possible_answers, query, VideoSegment):
4     # Answer is:
5     def execute_command(video, possible_answers, query):
6         video_segment = VideoSegment(video)
7         # Reason step-by-step
8         video_segment = video_segment.trim(0, video_segment.num_frames // 4) # Because of "at the
9         start"
10        man_segment = video_segment.object_sampling("man")
11        forward_backward_segment = man_segment.general_sampling("Is the man moving forward and
12        backward?")
13        baby_segment = forward_backward_segment.object_sampling("baby")
14        # create context dictionary
15        image_caption = baby_segment.get_caption(baby_segment.num_frames // 2)
16        video_caption = baby_segment.get_caption()
17        context = {
18            "Description of a scene when the man moves forward and backward": image_caption,
19            "Description of the video segment ": video_caption
20        }
21        # Answer the query
22        answer = baby_segment.language_query(context, query, possible_answers)
23        return answer

```

Listing B.3: Descriptive Reasoning Example in NeXT-QA dataset